

[← Voltar para o blog](#)

Python

PyGame do Python – Como Criar Jogos no Python



Pesquisar por...

**Matricule-se**

⚡ EM RESUMO

Conheça a biblioteca PyGame do Python e aprenda a criar jogos 2D do zero: instalação, janela, loop principal e elementos do jogo. Aula completa.

Conheça a biblioteca PyGame do Python e aprenda como criar jogos no Python a partir do zero nesta aula completa!

O **PyGame** é uma biblioteca do Python para criar jogos 2D. Com ela, você desenha telas, formas e imagens, controla teclado e mouse e cria o loop principal do jogo. Para começar, instale com `pip install pygame`, importe a biblioteca, inicialize com `pygame.init()` e monte o loop que atualiza a tela.

Caso prefira esse conteúdo no formato de vídeo-aula, assista ao vídeo abaixo ou acesse o nosso [canal do YouTube](#)!





O que você aprende neste vídeo

Resposta rápida: Nesta aula completa, o instrutor ensina a criar o seu primeiro jogo no Python com a biblioteca **PyGame**: o clássico Brick Breaker. Ele instala e inicializa o PyGame, configura a tela, cria os elementos (bola, jogador e blocos) como retângulos, monta o loop infinito do jogo, programa a movimentação e a colisão, e no fim mostra como personalizar tudo (tamanhos, cores, número de blocos).

Neste vídeo (69 min):

- [2:00](#) — Instalar e inicializar: `pip install pygame` e `pygame.init()`.
- [4:00](#) — Configurar a tela: `set_mode` (800×800) e `set_caption` (o título do jogo).
- [5:00](#) — Criar os elementos como retângulos (`pygame.Rect`): bola, jogador e blocos, com posição e tamanho.
- [13:00](#) — Dicionário de cores no padrão RGB (vermelho, verde, azul de 0 a 255).
- [20:00](#) — O loop infinito do jogo: verificar os eventos a cada instante (como o botão de fechar).
- [24:00](#) — Desenhar os elementos na tela; a lógica que posiciona cada bloco em grade.



- [44:00](#) — Movimentar o jogador (a barrinha) pelo teclado.
- [52:00](#) — Movimentar a bola e fazer ela quicar nas bordas e no jogador.
- [64:00](#) — Colisão da bola com os blocos, a pontuação e o fim de jogo.

Trechos do vídeo:

- O instrutor explica a estrutura de qualquer jogo no PyGame: você inicializa o jogo, cria os elementos e desenha na tela, e então roda um **loop infinito** — que a cada instante verifica a pontuação, o que aparece na tela e o que o usuário fez (tecla ou mouse).
- Cada elemento (bola, jogador, blocos) é criado como um retângulo do PyGame (`pygame.Rect`), com posição (distância da esquerda e do topo) e tamanho — e não uma simples tupla, porque o retângulo tem métodos próprios, como o de detectar colisão (`colliderect`).
- Ele deixa tudo parametrizado em variáveis (tamanho da tela, da bola, do jogador, número de blocos e de linhas) e monta um dicionário de cores no padrão RGB — assim dá para personalizar o jogo inteiro só mudando esses valores.

Para receber por e-mail o(s) arquivo(s) utilizados na aula, preencha:

PyGame do Python – Como Criar Jogos no Python

Na aula de hoje, quero te ensinar como criar jogos no Python através da biblioteca PyGame!



Você irá aprender, passo a passo, como criar um jogo de destruir blocos, passando pelas etapas de criação da tela, criação dos elementos, movimentação do usuário, regras, pontuação e execução do jogo.

Vamos ver como realizar cada um desses procedimentos, que são fundamentais para que o jogo funcione corretamente. Ao final, você terá um jogo totalmente funcional criado do zero usando Python.

Então, faça o download do material disponível e venha comigo aprender a criar jogos no Python!

Biblioteca PyGame

A PyGame é uma biblioteca completa, com diversos módulos em Python, projetada para a criação de jogos.

Sendo uma biblioteca de código aberto, ela é principalmente usada para o desenvolvimento de jogos 2D, mas também pode ser utilizada para outras aplicações multimídia, como a manipulação de imagens e a reprodução de áudio.

Ao longo desta aula, construiremos um jogo passo a passo, mas você pode sempre consultar a **documentação oficial da biblioteca** para ver outros exemplos, tutoriais e todas as funcionalidades disponíveis.

- <https://www.pygame.org/docs/>

Geralmente, a construção de jogos é feita utilizando **Programação Orientada a Objetos** (POO). Porém, para tornar o tutorial mais acessível para quem ainda não teve contato com POO, vamos desenvolver esse jogo sem entrar em orientação a objetos.

No entanto, caso queira entender e saber como funciona a programação orientada a objetos, vou deixar a aula abaixo como sugestão para você:



- [Programação Orientada a Objetos – POO](#)

Instalação e Importação da Biblioteca PyGame

Para começar o desenvolvimento do nosso jogo, o primeiro passo será instalar a biblioteca PyGame através do comando: **pip install pygame**.

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SQL CONSOLE
PS C:\Users\diego\OneDrive\Área de Trabalho\Python> pip install pygame
```

Feita a instalação, podemos importar a biblioteca e começar a utilizá-la.

```
PYTHON  Copiar
import pygame
```

O que Vamos Construir – Estrutura de Construção de Jogos

De modo geral, para construir um jogo precisamos seguir algumas etapas na seguinte ordem:

- Inicializar o jogo
- Construir as funções do jogo
- Desenhar os elementos na tela



- Criar um loop infinito para manter o jogo em execução

PYTHON

 Copiar

```
import pygame

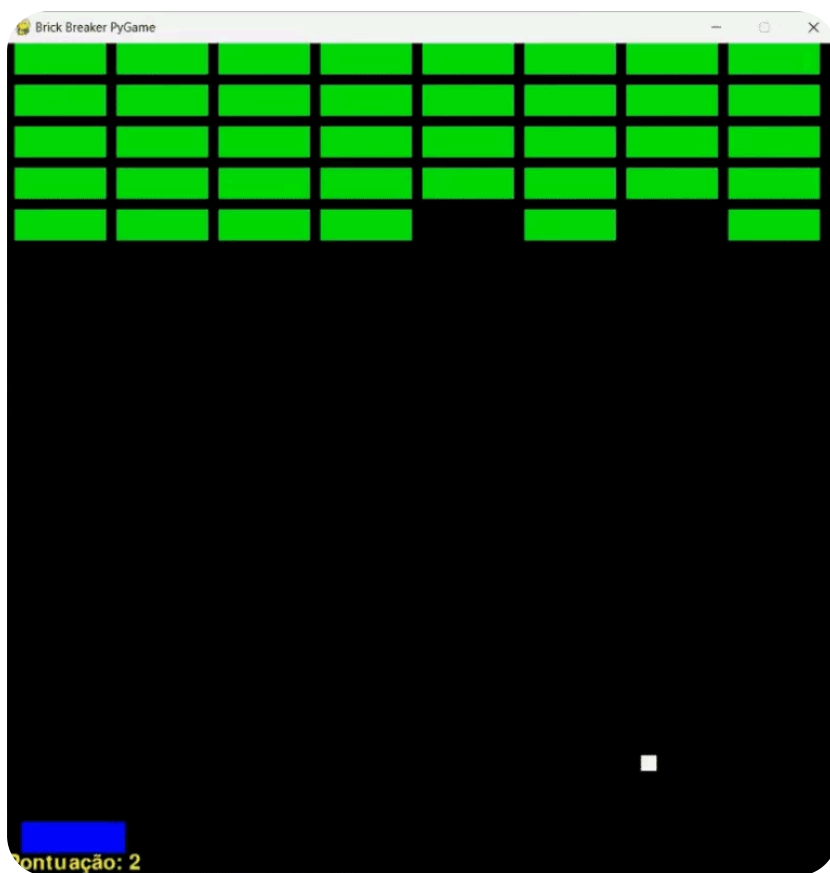
# inicializar

# desenhar os elementos na tela

# criar as funções do jogo

# criar um loop infinito
```

Seguindo cada um desses passos, vamos construir um jogo de destruir blocos, um **Brick Breaker**.



Inicializando o PyGame – Configurações

Para inicializar o PyGame, após a importação, precisamos executar o comando **init()**.

PYTHON Copiar

```
import pygame

# inicializar
pygame.init()
```

Feito isso, vamos criar as configurações básicas do nosso jogo, como tamanho da tela, tamanho da bola, jogador, quantidade de blocos, entre outras.

Começando pela tela, vamos definir o tamanho dela dentro de uma variável e passar essa variável como argumento para a função **set_mode** do módulo **display**. Dessa forma, teremos configurado a base da nossa tela.

É importante ressaltar que o tamanho definido para a tela não pode ser maior do que a tela do seu computador. Para o nosso jogo, um tamanho de **800×800** será o suficiente.

Além da tela, todo jogo precisa de um título que será exibido no nome da janela. Para definir o nome do jogo, utilizaremos a função **set_caption** e passaremos como argumento o nome desejado. No meu caso, será **Brick Breaker Youtube**.

Feito isso, precisamos configurar os demais elementos principais presentes no jogo: a bola, o jogador e os blocos.

Considerando que a tela tem 800 pixels, as medidas que funcionaram para a bola e o jogador foram de 15×15 para a bola e de 100 pixels para o retângulo do jogador.



Os blocos são definidos em cinco linhas com oito blocos em cada, totalizando 40 blocos no jogo.

PYTHON

 Copiar

```
import pygame

# inicializar
pygame.init()
tamanho_tela = (800, 800)
tela = pygame.display.set_mode(tamanho_tela)
pygame.display.set_caption("Brick Breaker Youtube")
tamanho_bola = 15
tamanho_jogador = 100
qtde_blocos_linha = 8
qtde_linhas_blocos = 5
qtde_total_blocos = qtde_blocos_linha * qtde_linhas_blocos
```

Criando os Elementos Bola e Jogador

Com as variáveis e medidas definidas para cada um dos elementos, podemos criá-los efetivamente.

Cada elemento do jogo é representado por um retângulo simples, então utilizaremos a classe **Rect** do Pygame para criar cada um deles.

Essa classe recebe como argumentos a posição com relação ao lado esquerdo (left), a posição com relação ao topo (top), a largura e a altura do elemento.

Para a altura e largura da bola, vamos utilizar a variável **tamanho_bola**. Já para o jogador, vamos passar a largura como sendo o **tamanho_jogador**, mas a altura vamos definir como **15 pixels**, para que não fique um quadrado.



PYTHON

 Copiar

```
import pygame

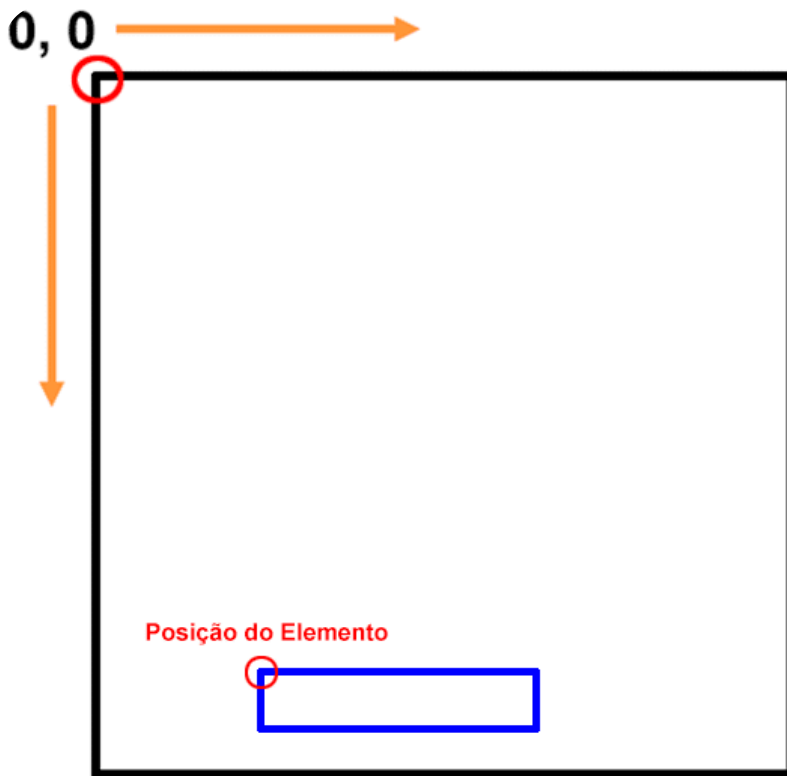
# inicializar
pygame.init()
tamanho_tela = (800, 800)
tela = pygame.display.set_mode(tamanho_tela)
pygame.display.set_caption("Brick Breaker Youtube")
tamanho_bola = 15
bola = pygame.Rect(100, 500, tamanho_bola, tamanho_bola)
tamanho_jogador = 100
jogador = pygame.Rect(0, 750, tamanho_jogador, 15)
```

Para que isso fique mais claro, observe a figura abaixo. O quadrado representa a nossa tela. O Pygame considera o ponto superior esquerdo como **coordenadas (0, 0)**.

Então, quando criamos um elemento e passamos as posições **left** (esquerda) e **top** (topo), estamos definindo a distância em pixels que esse elemento deve ser criado a partir do ponto (0, 0) na tela.

Feito isso, o PyGame considerará essa posição como o ponto inicial do elemento para criá-lo a partir da largura e altura fornecidas.





Função Para Criação dos Blocos

Para a criação dos blocos no jogo, teremos um pouco mais de trabalho, pois precisamos considerar que cada um deles terá uma posição distinta na tela.

O objetivo é preencher todos os blocos na tela, considerando a quantidade de blocos por linha e a quantidade de linhas.

Para definir essa lógica, vamos criar uma função que receberá como parâmetros a quantidade de blocos por linha e a quantidade de linhas de blocos.

Dentro da função, vamos declarar as dimensões da tela (**altura_tela**, **largura_tela**). Podemos acessar esses valores através da nossa tupla **tamanho_tela**, passando os índices correspondentes a cada medida.

Feito isso, precisamos determinar a largura e a altura de cada bloco. Por linha, queremos ter 8 blocos. Então, para calcular a largura de cada bloco, basta dividir a largura da tela (**largura_tela**) por 8.



Porém, se fizermos isso, os blocos ficarão grudados, e não é isso que queremos. Então, antes de calcular a largura do bloco, vamos definir a distância entre os blocos como sendo 5px.

Dessa forma, a **largura do bloco** será a largura da tela dividida por oito, menos a distância entre os blocos.

Para a altura dos blocos, podemos definir um valor padrão, como 15px. E, para calcular a distância entre as linhas, podemos somar a altura do bloco ao espaçamento que desejamos. Assim, evitamos criar uma linha sobre a outra ou uma colada na outra.

PYTHON

 Copiar

```
def criar_blocos(qtde_blocos_linha, qtde_linhas_blocos):  
    altura_tela = tamanho_tela[1]  
    largura_tela = tamanho_tela[0]  
    distancia_entre_blocos = 5  
    largura_bloco = largura_tela / 8 - distancia_entre_blocos  
    altura_bloco = 15  
    distancia_entre_linhas = altura_bloco + 10
```

Com essas variáveis definidas, nossa função irá inicializar uma lista de blocos vazia e, a partir disso, irá iterar sobre a quantidade de linhas e a quantidade de blocos que definimos anteriormente para criar cada bloco individualmente e adicioná-lo à lista.

Para a criação dos blocos, utilizaremos um loop aninhado com dois **for** para iterar sobre a quantidade de linhas e a quantidade de blocos por linha. O loop externo controlará as linhas e o loop interno a quantidade de blocos dentro de cada uma.

Dentro do loop interno, criaremos cada bloco utilizando a classe **Rect**. As coordenadas left e top para cada bloco serão calculadas da seguinte forma:

- **Left:** Será calculada como o índice i multiplicado pela largura do bloco mais a distância entre eles. Isso posicionará o primeiro bloco (i=0) ao lado esquerdo



tela, e os demais blocos à direita do anterior, respeitando a distância entre eles.

- **Top:** A posição vertical dos blocos será calculada como o índice j multiplicado pela distância entre as linhas. Isso posicionará cada linha de blocos abaixo da anterior.

Para a largura e a altura de cada bloco, utilizaremos as variáveis criadas no início da função.

Após criar o bloco, a função irá adicioná-lo à lista `blocos` com o método [append](#). Após todos os blocos serem criados e adicionados à lista, a função retornará a lista **`blocos`**.

PYTHON

 Copiar

```
def criar_blocos(qtde_blocos_linha, qtde_linhas_blocos):
    altura_tela = tamanho_tela[1]
    largura_tela = tamanho_tela[0]
    distancia_entre_blocos = 5
    largura_bloco = largura_tela / 8 - distancia_entre_blocos
    altura_bloco = 15
    distancia_entre_linhas = altura_bloco + 10
    blocos = []

    # criar os blocos
    for j in range(qtde_linhas_blocos):
        for i in range(qtde_blocos_linha):
            # criar o bloco
            bloco = pygame.Rect(i * (largura_bloco + distancia_entre_blocos), j * dis
            # adicionar o bloco na lista de blocos
            blocos.append(bloco)
    return blocos
```

Configurando as Cores do Jogo



Com os elementos configurados, precisamos definir as cores do nosso jogo. As cores no PyGame seguem o **padrão RGB** (Red, Green e Blue ou Vermelho, Verde e Azul), variando de **0 a 255**.

Esses valores indicam a quantidade de vermelho, verde e azul presente naquela cor. Diferentes combinações nos permitem criar diferentes cores. Por exemplo, (255, 255, 255) representa a cor branca, enquanto (0, 0, 0) representa a cor preta.

Para definir as cores do nosso jogo, vamos criar um dicionário de cores em que a chave será o nome da cor e o valor será a tupla contendo a quantidade de vermelho, verde e azul.

PYTHON

 Copiar

```
cores = {  
    "branca": (255, 255, 255),  
    "preta": (0, 0, 0),  
    "amarela": (255, 255, 0),  
    "azul": (0, 0, 255),  
    "verde": (0, 255, 0)  
}
```

Variáveis Importantes – Funcionamento do Jogo

Para encerrar essa etapa de configuração e inicialização, precisamos definir algumas variáveis fundamentais para o funcionamento do nosso jogo: fim_jogo, pontuacao e movimento_bola.

A variável **fim_jogo** indica se o jogo terminou ou não. Ela inicia como False e, quando o jogo acabar, dentro do loop de execução, alteramos o valor dessa variável para True.

A **pontuacao** armazena a pontuação do jogador, que inicia em 0.

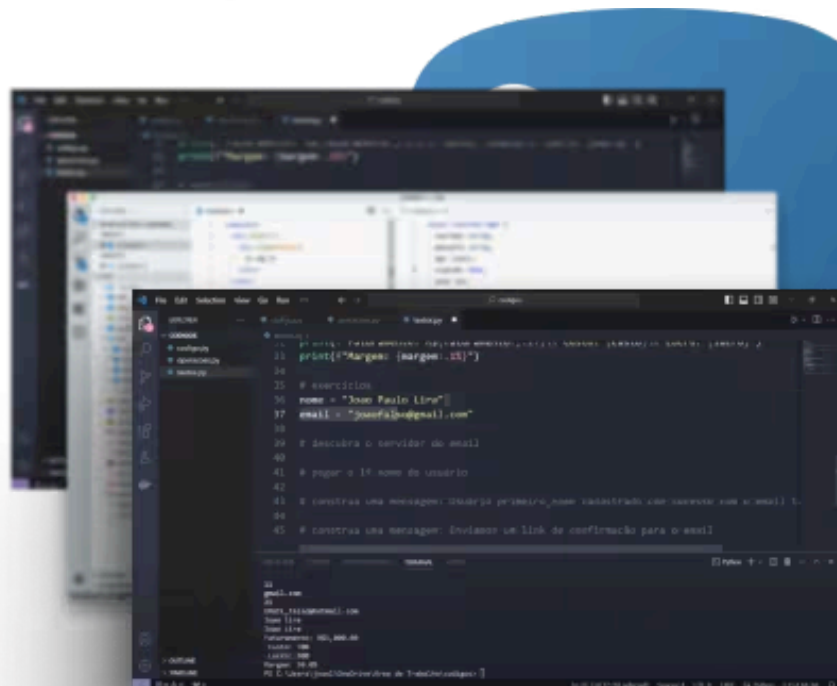


E a variável **movimento_bola** será uma lista contendo dois valores que representam a velocidade de movimento da bola em pixels.



Você vai aprender a linguagem de Programação que mais cresce no Mundo para fazer automações incríveis, desenvolver sites, fazer análises de dados, trabalhar com Ciência de Dados, Inteligência Artificial, mesmo que você nunca tenha tido nenhum contato com Programação na vida.

Começar agora →



O primeiro valor indica o movimento horizontal (da esquerda para a direita) e o segundo valor indica o movimento vertical (de cima para baixo).

Como esses valores serão alterados ao longo do jogo, eles precisam ser definidos como uma lista (mutável) e não como uma tupla (imutável).



```
fim_jogo = False  
pontuacao = 0  
movimento_bola = [7, -7]
```

Criar o Loop Infinito – Execução do Jogo

Para que possamos começar a desenhar e visualizar os elementos na tela, primeiro precisamos criar o loop infinito que irá executar e manter o nosso jogo aberto até que uma condição de término seja atendida, como o jogador fechar a janela do jogo.

Esse loop **while** será executado enquanto a variável **fim_jogo for False**. Quando o valor dessa variável se tornar True, o loop será interrompido e o jogo encerrado.

Dentro desse loop, utilizaremos um for para iterar sobre todos os eventos capturados pelo PyGame utilizando o método **pygame.event.get()**. Esse método retorna uma lista com todos os eventos que ocorreram desde a última iteração do loop.

Se algum desses eventos for do tipo **pygame.QUIT**, que indica que o jogador clicou para fechar a janela, a variável fim_jogo será alterada para True, o que encerrará o loop na próxima iteração.

Para garantir a atualização da tela com todas as mudanças feitas desde a última alteração e controlar a velocidade em que essa atualização é feita, utilizaremos o método **flip** e a função **wait**.

A função wait do módulo time recebe o valor em milissegundos que o jogo deve aguardar a cada iteração do loop. Isso indica o intervalo de tempo que ele deve aguardar para verificar os eventos que ocorreram nele.

Já o método flip é o responsável por atualizar a tela com as mudanças feitas desde a última atualização. Ele é necessário para garantir que qualquer alteração visual do jogo seja exibida para o jogador.



Por fim, após o término desse loop principal, definimos o comando **pygame.quit()** para encerrar o PyGame corretamente.

PYTHON

 Copiar

```
# criar um loop infinito
while not fim_jogo:
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            fim_jogo = True
    pygame.time.wait(1)
    pygame.display.flip()
pygame.quit()
```

Desenhando a Tela e os Elementos do Jogo

Agora que temos o loop responsável por executar e manter o nosso jogo aberto, podemos desenhar a tela e os demais elementos do jogo.

Para desenhar a tela, vamos utilizar o método **fill**, passando como argumento a cor desejada. Nesse caso, utilizaremos o dicionário de cores para preencher nossa tela com um fundo preto.

O jogador e a bola podem ser desenhados utilizando a função **rect** do módulo **draw** do PyGame. Para essa função, passaremos a tela onde o elemento deve ser desenhado, a cor dele e qual é esse elemento. Isso criará a tela inicial do nosso jogo.

O ideal, quando estamos trabalhando com jogos, é fazer a programação orientada a objetos (POO). Nesse caso, mesmo sem estarmos seguindo a abordagem de POO, podemos estruturar cada elemento em funções separadas.



Dessa forma, mantemos a estrutura do nosso código mais organizada, facilitando futuras edições, ajustes e a manutenção dele.

Então, iremos definir toda essa criação inicial dentro da função **desenhar_inicio_jogo**.

PYTHON

 Copiar

```
# desenhar os elementos na tela
def desenhar_inicio_jogo():
    tela.fill(cores["preta"])
    pygame.draw.rect(tela, cores["azul"], jogador)
    pygame.draw.rect(tela, cores["branca"], bola)
```

Para desenhar os blocos, vamos criar uma função chamada **desenhar_blocos** que receberá a lista blocos que criamos anteriormente e, para cada bloco presente dentro da lista, irá desenhar cada um deles usando a função rect.

PYTHON

 Copiar

```
# desenhar os elementos na tela
def desenhar_inicio_jogo():
    tela.fill(cores["preta"])
    pygame.draw.rect(tela, cores["azul"], jogador)
    pygame.draw.rect(tela, cores["branca"], bola)

def desenhar_blocos(blocos):
    for bloco in blocos:
        pygame.draw.rect(tela, cores["verde"], bloco)
```

Feito isso, antes do loop de execução do jogo, vamos chamar a função criar_blocos e, dentro do loop, vamos chamar desenhar_inicio_jogo e desenhar_blocos.



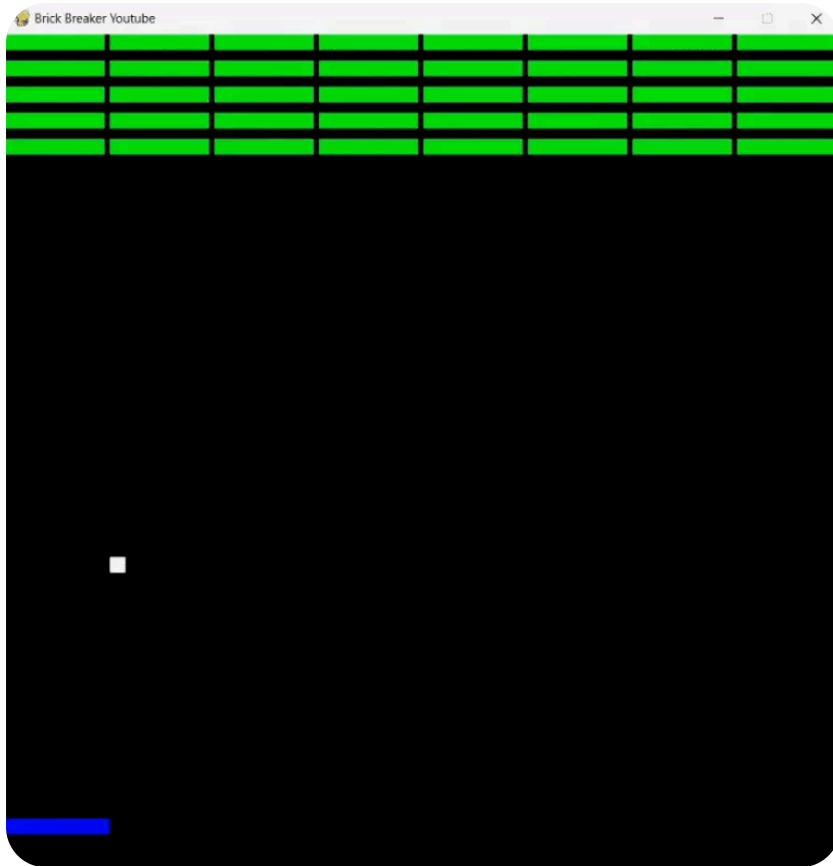
Isso porque os blocos só precisam ser criados uma única vez; no entanto, a tela com o jogador e a bola, e o desenho dos blocos na tela, precisam ser desenhados a cada interação do jogador e atualização da tela do jogo.

PYTHON Copiar

```
blocos = criar_blocos(qtde_blocos_linha, qtde_linhas_blocos)
# criar um loop infinito
while not fim_jogo:
    desenhar_inicio_jogo()
    desenhar_blocos(blocos)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            fim_jogo = True
    pygame.time.wait(1)
    pygame.display.flip()
pygame.quit()
```

Se executarmos nosso jogo agora, teremos todos os elementos dele sendo exibidos na tela.





Criando as Regras do Jogo – Movimento do Jogador

A próxima etapa de criação do nosso jogo será definir as regras dele. Essas regras envolvem principalmente as movimentações da bola e do jogador.

Quando a bola colide com o jogador, temos uma mudança de posição dela. Quando colide com o bloco, temos uma mudança de posição e uma eliminação do bloco.

A movimentação do jogador é controlada pelas teclas direcionais direita e esquerda, único movimento que nosso jogador pode fazer.

Para isso, vamos criar uma função chamada **movimentar_jogador** e utilizar os eventos do PyGame para capturar qual tecla foi pressionada e calcular a movimentação feita.

A função `movimentar_jogador` receberá como parâmetro um evento do PyGame. Dentro dela, vamos verificar se o evento recebido é do tipo **KEYDOWN**, indicando que



uma tecla foi pressionada.

Caso seja um evento KEYDOWN, iremos verificar, utilizando a função `key`, se a tecla pressionada foi a seta para a direita (**K_RIGHT**) ou para a esquerda (**K_LEFT**).

Para cada um desses movimentos, precisamos definir uma lógica específica.

Movimentação para a Direita:

Caso o jogador esteja se movimentando para a direita, precisamos verificar se a posição no eixo x dele (**jogador.x**), somada ao seu tamanho, não ultrapassa a largura da tela. Caso não ultrapasse, a posição do jogador é aumentada em 5 pixels para a direita.

Movimentação para a Esquerda:

Por outro lado, se o jogador estiver se movendo para a esquerda, vamos verificar se a posição dele é maior do que 0, garantindo que ele não ultrapasse o limite esquerdo da tela. Caso seja, a posição do jogador é diminuída em 5 pixels.

Essas verificações em ambos os movimentos garantem que o jogador não ultrapasse o limite da tela, delimitando a área do jogo.

PYTHON Copiar

```
# criar as funções do jogo
def movimentar_jogador(evento):
    if evento.type == pygame.KEYDOWN:
        if evento.key == pygame.K_RIGHT:
            if (jogador.x + tamanho_jogador) < tamanho_tela[0]:
                jogador.x = jogador.x + 5
        if evento.key == pygame.K_LEFT:
            if jogador.x > 0:
                jogador.x = jogador.x - 5
```



Com essa função criada, podemos adicioná-la dentro do loop de execução do jogo.

PYTHON

 Copiar

```
blocos = criar_blocos(qtde_blocos_linha, qtde_linhas_blocos)
# criar um loop infinito
while not fim_jogo:
    desenhar_inicio_jogo()
    desenhar_blocos(blocos)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            fim_jogo = True
    movimentar_jogador(evento)
    pygame.time.wait(1)
    pygame.display.flip()
pygame.quit()
```

Criando as Regras do Jogo – Movimento e Colisão da Bola

Diferente do jogador, a bola deve se movimentar a cada execução do loop principal. Ou seja, a cada milissegundo em que nosso jogo é atualizado, a bola deve estar em movimento, não precisando de um evento para acontecer.

A função responsável por controlar o movimento da bola será a **movimentar_bola**. Essa função recebe como argumento a bola criada e retorna o novo movimento da bola. Dentro dela, iremos atualizar a posição da bola e lidar com as colisões possíveis.

No início dessa função, vamos definir a variável movimento, que vai armazenar a direção e a velocidade da bola nos eixos x (horizontal) e y (vertical). Inicialmente, essa lista receberá os valores que definimos em **movimento_bola**.



Em seguida, essa função deverá atualizar a posição da bola nos eixos x e y, somando os valores de movimento aos seus valores atuais.

PYTHON

 Copiar

```
def movimentar_bola(bola):  
    movimento = movimento_bola  
    bola.x = bola.x + movimento[0]  
    bola.y = bola.y + movimento[1]
```

Feito isso, podemos verificar as colisões da bola com os limites da tela.

Colisões Laterais – Borda Esquerda e Direita:

As colisões laterais invertem o movimento da bola no eixo x, o movimento horizontal dela.

Para determinar a colisão com a parede esquerda, vamos verificar se a posição x da bola (**bola.x**) é menor ou igual a 0. Na parede direita, verificaremos se a posição x da bola somada ao tamanho dela (**tamanho_bola**) é maior ou igual à largura da tela.

Caso qualquer uma dessas verificações seja verdadeira, o movimento horizontal da bola (**movimento[0]**) passará a ser invertido.

Colisões Verticais – Borda Superior e Inferior:

Para as colisões verticais, teremos dois casos distintos. Uma colisão com a borda superior resulta na inversão do movimento vertical da bola. Já uma colisão com a parte inferior da tela resultará em um fim de jogo.

Para determinar a colisão superior, vamos verificar se a posição y da bola (**bola.y**) é menor ou igual a 0. Caso seja, a bola terá seu movimento vertical (**movimento[1]**)



invertido.

Na parte inferior da tela, verificaremos se a posição y da bola somada ao seu tamanho é maior ou igual à altura da tela. Se essa verificação for verdadeira, o movimento será definido como **None**, que será uma das formas de encerrarmos o jogo.

PYTHON Copiar

```
def movimentar_bola(bola):  
    movimento = movimento_bola  
    bola.x = bola.x + movimento[0]  
    bola.y = bola.y + movimento[1]  
  
    if bola.x <= 0:  
        movimento[0] = - movimento[0]  
    if bola.y <= 0:  
        movimento[1] = - movimento[1]  
    if bola.x + tamanho_bola >= tamanho_tela[0]:  
        movimento[0] = - movimento[0]  
    if bola.y + tamanho_bola >= tamanho_tela[1]:  
        movimento = None
```

Colisão com o Jogador:

Além das colisões com as bordas da tela, a bola também pode e deve colidir com o jogador. Quando isso acontece, o movimento vertical da bola deve ser invertido.

Para determinar a colisão entre a bola e o jogador, usamos o método **collidepoint** do retângulo do jogador para verificar se a posição atual da bola coincide com a posição atual do jogador.

Se a colisão for confirmada, o movimento vertical da bola (**movimento[1]**) deverá ser invertido, permitindo que o jogador rebata a bola em direção aos blocos.



PYTHON

 Copiar

```
def movimentar_bola(bola):  
    movimento = movimento_bola  
    bola.x = bola.x + movimento[0]  
    bola.y = bola.y + movimento[1]  
  
    if bola.x <= 0:  
        movimento[0] = - movimento[0]  
    if bola.y <= 0:  
        movimento[1] = - movimento[1]  
    if bola.x + tamanho_bola >= tamanho_tela[0]:  
        movimento[0] = - movimento[0]  
    if bola.y + tamanho_bola >= tamanho_tela[1]:  
        movimento = None  
  
    if jogador.collidepoint(bola.x, bola.y):  
        movimento[1] = - movimento[1]
```

Colisões com os Blocos:

Por fim, precisamos determinar a colisão da bola com os blocos, que é o objetivo principal do nosso jogo.

Quando a bola colide com um bloco, além de ter seu movimento vertical invertido, o bloco atingido deve ser removido.

Para fazermos isso, vamos iterar sobre todos os blocos e utilizar novamente o método **collidepoint** para verificar a colisão.

Se a colisão for detectada, removemos o bloco da lista de blocos e invertamos o movimento vertical da bola.

PYTHON

 Copiar


```
def movimentar_bola(bola):  
    movimento = movimento_bola  
    bola.x = bola.x + movimento[0]  
    bola.y = bola.y + movimento[1]  
  
    if bola.x <= 0:  
        movimento[0] = - movimento[0]  
    if bola.y <= 0:  
        movimento[1] = - movimento[1]  
    if bola.x + tamanho_bola >= tamanho_tela[0]:  
        movimento[0] = - movimento[0]  
    if bola.y + tamanho_bola >= tamanho_tela[1]:  
        movimento = None  
  
    if jogador.collidepoint(bola.x, bola.y):  
        movimento[1] = - movimento[1]  
  
    for bloco in blocos:  
        if bloco.collidepoint(bola.x, bola.y):  
            blocos.remove(bloco)  
            movimento[1] = - movimento[1]  
  
    return movimento
```

Com a função **movimentar_bola** criada, podemos integrá-la ao loop principal do jogo para garantir seu movimento adequado e que as colisões sejam tratadas a cada atualização.

PYTHON

 Copiar

```
# criar um loop infinito  
while not fim_jogo:  
    desenhar_inicio_jogo()  
    desenhar_blocos(blocos)  
    for evento in pygame.event.get():  
        if evento.type == pygame.QUIT:  
            fim_jogo = True
```



```
movimentar_jogador(evento)
movimento_bola = movimentar_bola(bola)

pygame.time.wait(1)
pygame.display.flip()

pygame.quit()
```

Regra para o Usuário Perder o Jogo

Nosso jogo já está praticamente pronto, só falta determinar as condições de fim de jogo ao perder ou atingir a pontuação máxima.

Como já definimos que ao tocar na parte inferior da tela o movimento da bola deve ser None. Vamos começar definindo a regra de derrota.

Dentro do loop principal, iremos verificar se não existe movimento_bola (**if not movimento_bola**), que é o mesmo que verificar se movimento_bola é igual a **None**.

Caso essa verificação seja verdadeira, a variável **fim_jogo** deve ser alterada para True, encerrando o loop e a execução do jogo.

PYTHON Copiar

```
# criar um loop infinito
while not fim_jogo:
    desenhar_inicio_jogo()
    desenhar_blocos(blocos)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            fim_jogo = True

    movimentar_jogador(evento)
```



```
movimento_bola = movimentar_bola(bola)

if not movimento_bola:
    fim_jogo = True

pygame.time.wait(1)
pygame.display.flip()

pygame.quit()
```

Pontuação e Vitória do Usuário

Agora, para finalizar o desenvolvimento do nosso jogo, só nos resta determinar a condição de vitória do jogador e calcular a pontuação feita por ele. Para isso, vamos criar a função **atualizar_pontuacao**.

Essa função receberá como parâmetro a pontuação (**pontuacao**) e exibirá na tela o valor atual da pontuação do jogador.

Para isso, utilizaremos a classe **Font** do módulo font do Pygame para definir o nome da fonte e o tamanho dela.

Em seguida, vamos usar o método **render** da nossa fonte para criar o texto que deve ser exibido na tela, passando como argumentos o texto desejado, o número 1 para o parâmetro antialias e a cor.

Por fim, através do método **blit**, vamos definir que queremos mostrar o texto criado e em qual posição da tela desejamos colocá-lo.

Além disso, se a pontuação do jogador for maior ou igual à quantidade total de blocos, essa função deverá retornar True, caso contrário retornará False.



PYTHON

 Copiar

```
def atualizar_pontuacao(pontuacao):  
    fonte = pygame.font.Font(None, 30)  
    texto = fonte.render(f"Pontuação: {pontuacao}", 1, cores["amarela"])  
    tela.blit(texto, (0, 780))  
    if pontuacao >= qtde_total_blocos:  
        return True  
    else:  
        return False
```

Dessa forma, ao atingir a pontuação máxima, conseguiremos encerrar o jogo dentro do loop de execução principal.

Podemos chamar a função **atualizar_pontuacao** armazenando o resultado dela na variável **fim_jogo**. Assim, quando a função retornar True porque todos os blocos foram destruídos, o jogo será encerrado.

Para calcular a pontuação do usuário, basta subtrairmos da quantidade total de blocos o tamanho atual da lista de blocos.

Como essa lista é atualizada cada vez que um bloco é destruído, a pontuação aumentará em 1 ponto para cada bloco.

PYTHON

 Copiar

```
# criar um loop infinito  
while not fim_jogo:  
    desenhar_inicio_jogo()  
    desenhar_blocos(blocos)  
    fim_jogo = atualizar_pontuacao(qtde_total_blocos - len(blocos))  
    for evento in pygame.event.get():  
        if evento.type == pygame.QUIT:  
            fim_jogo = True  
  
    movimentar_jogador(evento)
```



```
movimento_bola = movimentar_bola(bola)

if not movimento_bola:
    fim_jogo = True

pygame.time.wait(1)
pygame.display.flip()

pygame.quit()
```

Feito isso, você terá concluído a criação do seu jogo em Python com a biblioteca Pygame.

Testando o Jogo e Ajustando a Velocidade

Após concluir o código do jogo, é importante que você faça os testes e ajustes necessários para adequar a dificuldade e a jogabilidade com o que você espera.

Por exemplo, você pode aumentar ou diminuir a velocidade de movimento da bola modificando a variável **movimento_bola**.

PYTHON Copiar

```
movimento_bola = [2, -2]
```

O mesmo pode ser feito para o movimento do jogador, aumentando ou diminuindo o valor que é acrescido e retirado de **jogador.x** dentro da função **movimentar_jogador**.



PYTHON

 Copiar

```
def movimentar_jogador(evento):  
    if evento.type == pygame.KEYDOWN:  
        if evento.key == pygame.K_RIGHT:  
            if (jogador.x + tamanho_jogador) < tamanho_tela[0]:  
                jogador.x = jogador.x + 5  
        if evento.key == pygame.K_LEFT:  
            if jogador.x > 0:  
                jogador.x = jogador.x - 5
```

Entre outras mudanças e ajustes que você pode fazer, como o número de blocos, cores, e o que mais você julgar necessário.

Perguntas frequentes sobre o PyGame do Python

1. O que é o PyGame?

É uma biblioteca do Python voltada para a criação de jogos 2D e aplicações multimídia. Ela oferece recursos para desenhar gráficos, tocar sons, detectar colisões e capturar comandos do teclado e do mouse, sendo muito usada para aprender lógica de programação na prática.

2. Como instalar o PyGame?

Use o gerenciador de pacotes do Python: digite `pip install pygame` no terminal. Depois, importe no código com `import pygame` e inicialize com `pygame.init()`. Se houver mais de uma versão do Python instalada, garanta que o pip usado corresponda à versão correta.



3. Como criar um jogo simples com PyGame?

Inicialize o PyGame, crie a janela com `pygame.display.set_mode()`, monte um loop principal que processa eventos (como teclas e o fechar da janela), atualize as posições dos elementos, desenhe tudo na tela e chame `pygame.display.update()` a cada quadro do jogo.

4. Preciso saber Python para usar o PyGame?

Sim, é recomendável conhecer o básico de Python: variáveis, listas, laços de repetição e funções. O PyGame não substitui esses fundamentos — ele os aplica. Por isso, criar jogos simples é uma ótima forma de praticar lógica e fixar a linguagem.

5. O que é o loop principal de um jogo?

É o ciclo que mantém o jogo rodando: a cada repetição ele lê os comandos do jogador, atualiza a lógica (posições, pontuação, colisões) e redesenha a tela. Esse loop roda muitas vezes por segundo, criando a sensação de movimento contínuo.

Conclusão – PyGame do Python – Como Criar Jogos no Python

Nessa aula você aprendeu como usar a biblioteca PyGame para criar seu primeiro jogo em Python a partir do zero! Passando por todos os processos e etapas que envolvem a criação de um jogo no Python.

Caso queira se aprofundar mais nessa biblioteca e aprender como desenvolver outros jogos, vou deixar aqui o link para o nosso curso gratuito de criação de jogos com Python e também uma aula onde você pode aprender a construir o jogo Snake.

- [Curso Criação de Jogos com Python](#)
- [Jogo Snake em Python](#)



[Dicas e Truques de SQL para Iniciantes»](#)

Hashtag Treinamentos

Para acessar outras publicações de Python, [clique aqui!](#)

Quer aprender mais sobre Python com um minicurso básico gratuito?

*Seu melhor e-mail

QUERO GARANTIR MINHA VAGA

Posts mais recentes da Hashtag Treinamentos



DASHBOARDS AUTOMÁTICOS

no Gemini



The screenshot shows the Gemini Analytics dashboard with the following data:

FILTROS DINÂMICOS				
Ano	Mês	País	Produto	Forma de Pagamento
Todos os anos	Todos os meses	Todos os países	Todos os produtos	Cartão de crédito

METRICAS	VALORES
FATURAMENTO TOTAL	R\$ 7.750,00
VOLUME DE VENDAS	18
TICKET MÉDIO	R\$ 430,56
CLIENTES ÚNICOS	18
LÍDER DE RECEITA	Produto D

EVOLUÇÃO MENSAL POR ANO: Gráfico de linhas mostrando o faturamento de 2024, 2025 e 2026.

FATURAMENTO POR PAÍS: Gráfico de barras mostrando o faturamento por país (Espanha, Colombia, Argentina, Chile).

SEM PROGRAMAR



The screenshot shows the Lovable app interface on a smartphone. The app displays a 'Pix Recebido' notification for R\$2000.00. Below this, it shows a 'Conta' summary with a balance of R\$ 2.000,00 and a 'Meus gastos' section with a total of R\$ 1.214,00. At the bottom, it shows a 'Cartão de crédito' summary with a balance of R\$ 650,00 and a limit of R\$ 3.350,00.

Crie um Site Completo com IA: Projeto No Code de R\$ 2.000

Aprenda a criar um site completo com IA, sem programar, com agente de atendimento automático. Veja o passo a passo prático e gratuito.

16 de julho de 2026





Erros comuns que gestores cometem ao tentar aumentar a produtividade da equipe

HASHTAG TREINAMENTOS

PARA EMPRESAS

6 erros de gestão que travam a produtividade da equipe

Conheça os principais erros de gestão que travam a produtividade da equipe e o que fazer para corrigir cada um deles de forma prática.

16 de julho de 2026





PROCV com SE: Como Combinar as Funções no Excel

Aprenda a combinar PROCV com SE no Excel para calcular bonificações por desempenho, usando SE aninhado e buscando o salário em outra planilha com o PROCV.

16 de julho de 2026





Como publicar um site com Claude: guia completo do zero

Aprenda como publicar um site com Claude: crie no Claude Code, suba no GitHub, publique na Vercel com domínio próprio e banco de dados grátis no Supabase.

13 de julho de 2026



AUTOR DO ARTIGO

Diego Monutti

Analista de Operações e Especialista em Python



Analista de Operações e SEO, além de redator técnico na Hashtag Treinamentos, onde produz conteúdos sobre dados, IA, programação e produtividade. Pós-graduando em Ciência de Dados e Inteligência Artificial pela Universidade Tecnológica Federal do Paraná (UTFPR), com foco em Business Intelligence, Machine Learning, MLOps e Big Data.

[LinkedIn](#)[Ver mais posts →](#)**Hashtag Treinamentos**

Do Básico ao Avançado — Torne-se o funcionário promissor de qualquer empresa.

Verificador de Números da Equipe Hashtag - Confie apenas nos números que validar clicando nesse link aqui

Cursos

[Todos os cursos](#)[Comunidade
Impressionadora](#)[Excel](#)[Power BI](#)[Inteligência Artificial](#)[Análise de Dados](#)[Python](#)

Páginas

[Pós-graduação](#)[Para empresas](#)[Blog](#)[Contato](#)[Quem somos](#)[Depoimentos](#)[Instrutores](#)[Imprensa](#)

Regulamento

[Termos de Uso](#)[Política de Privacidade](#)

[Apostilas](#)

[Carreira em Dados](#)

[Mapa do site](#)

© 2026. Hashtag Treinamentos. Todos os direitos reservados.

